

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



CSE 4621: Microprocessor and Interfacing Assignment 1

Written by - Namisa Najah Raisa Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

July 20, 2025

Contents

1	Problem Statement	3
1.1	What is the problem?	3
1.2	Why is this important?	3
1.3	How will it be solved?	3
2	Technical Specifications	3
2.1	Hardware Components	3
2.2	Software Components	4
3	System Design and Architecture	4
3.1	Overview	4
3.2	Network Structure	5
3.3	Component Roles	5
4	Implementation Plan	6
4.1	Phase 1: Prototyping	6
4.2	Phase 2: Hub Integration	6
4.3	Phase 3: Deployment	6
4.4	Phase 4: Scaling	6
5	Conclusion	6

1 Problem Statement

1.1 What is the problem?

In urban environments, detecting emergencies such as accidents, falls, or medical crises in real-time is challenging due to reliance on manual reporting or limited CCTV coverage. Existing systems often fail when victims are unconscious or in isolated areas.

1.2 Why is this important?

Timely emergency response is critical for saving lives and preventing further harm in urban environments. I think current systems suffer from three major limitations: they depend on victims being conscious and able to call for help, they lack coverage in isolated areas, and they often provide insufficient location accuracy. My solution addresses these gaps by creating an automated detection network that operates 24/7, requires no victim participation, and pinpoints emergencies with GPS precision. The system's value proposition lies in its ability to reduce response times during life-threatening situations - scenarios where every second counts.

1.3 How will it be solved?

A network of microcontroller-based sensor nodes (ATMega16/Arduino) will monitor environmental and behavioral anomalies (e.g., loud noises, prolonged immobility). Detected emergencies are relayed to a central hub (Raspberry Pi) for validation and alert dissemination via SMS, email, and dashboards.

2 Technical Specifications

2.1 Hardware Components

- **Microcontrollers**
 - **ATMega16**: 16 KB Flash, 32 I/O pins, 8-channel ADC for sensor inputs.
 - **Arduino Uno**: Alternative for prototyping; ATmega328P with USB support.
- **Sensors**
 - **SW-420 Vibration Sensor**: Detects falls/impacts.
 - **HC-SR04 Ultrasonic Sensor**: Measures distance to identify immobility.
 - **DHT11**: Monitors temperature/humidity for environmental context.
 - **Sound Sensor**: Captures distress signals (e.g., screams).
 - **NEO-6M GPS**: Provides exact incident coordinates via USART.
- **Communication**

- **Wi-Fi/GSM Modules:** Enable data transmission to the hub.
- **Raspberry Pi:** Acts as the gateway, aggregating data and triggering alerts.
- **Power & Enclosure**
 - 5V supply with 10,000mAh backup battery.
 - Weatherproof casing for outdoor durability.

2.2 Software Components

- **Embedded C/Arduino IDE:** For sensor data processing on nodes.
- **TensorFlow Lite:** Optional edge-based anomaly detection (e.g., abnormal sound patterns).
- **MQTT/HTTP:** Protocols for node-to-hub communication.
- **Flask Dashboard:** Real-time visualization of emergencies on a map.
- **REST API:** Integrates with emergency services (police, hospitals).

3 System Design and Architecture

3.1 Overview

The system comprises distributed sensor nodes (ATMega16/Arduino) and a centralized Raspberry Pi hub. Nodes process data locally to minimize latency, while the hub validates alerts and dispatches them via multiple channels (SMS, webhooks, APIs).

3.2 Network Structure

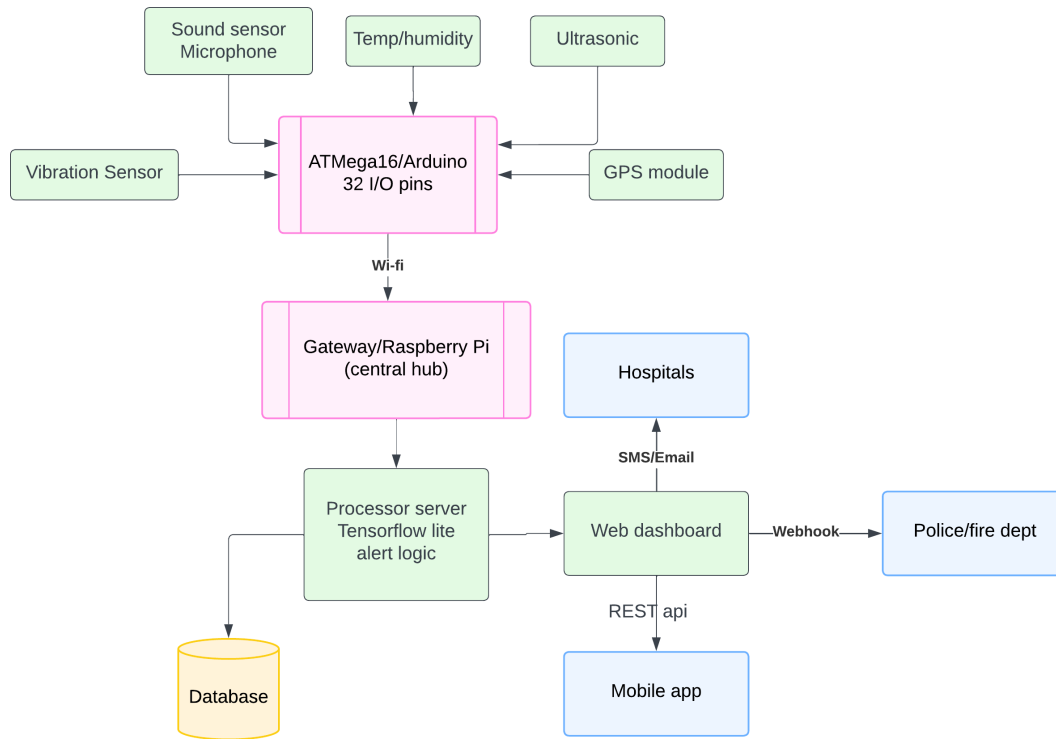


Figure 1: System Architecture Diagram showing data flow from sensors to emergency services.

Figure 1 illustrates the data flow from sensors → microcontrollers → Raspberry Pi hub → emergency services and dashboards.

3.3 Component Roles

- **Sensor Nodes**

- Collect and preprocess data (e.g., vibration thresholds, sound decibels).
- Transmit filtered alerts to the hub via Wi-Fi/USART.

- **Central Hub (Raspberry Pi)**

- Runs alert logic (e.g., cross-referencing GPS with sensor data).
- Updates databases and notifies responders via SMS/email.

- **Communication**

- **USART**: For wired node connections.

- **MQTT**: Lightweight messaging for wireless nodes.
- **User Interfaces**
 - **Web Dashboard**: Displays live alerts and locations.
 - **Mobile App**: Provides push notifications to responders.

4 Implementation Plan

4.1 Phase 1: Prototyping

- Assemble and test sensor nodes with ATmega16/Arduino.
- Develop embedded firmware for sensor data acquisition and filtering.

4.2 Phase 2: Hub Integration

- Configure Raspberry Pi to receive MQTT/USART messages.
- Implement Flask dashboard with map integration (Google Maps API).

4.3 Phase 3: Deployment

- Install nodes in high-risk areas (alleys, parks).
- Optimize power management for 24/7 operation.

4.4 Phase 4: Scaling

- Add LoRa for long-range communication in sprawling areas.
- Integrate cloud services (AWS IoT) for city-wide deployments.

5 Conclusion

The **Urban Emergency Detection System** leverages cost-effective microcontrollers and IoT protocols to bridge gaps in urban safety. Future enhancements include AI-based false-positive reduction and solar-powered nodes for sustainability.